Real-Time FPGA Implementation of a QPSK Modulator and Demodulator with Simulated AWGN Channel

Anonymous CVPR submission

Paper ID MTR16

Abstract

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique which transmits two bits per symbol without additional bandwidth. This project implements a complete, real-time QPSK transmitter and receiver entirely on a Spartan-7 FPGA (Boolean Board, XC7S50). The system performs bit-to-symbol mapping, carrier generation, quadrature mixing, and matched low-pass filtering, and includes a simulated AWGN channel with live-adjustable noise level for testing. System performance is validated through bit-error-rate (BER) measurement and live constellation visualization streamed over UART. This report defines the problem statement, reviews relevant literature on QPSK theory and existing FPGA-based QPSK implementations, and outlines the architecture and verification plan for the mid-term and final phases. Carrier recovery via a Costas loop, symbol timing recovery via the Gardner algorithm, and a coded-versus-uncoded BER comparison are being considered as optional extensions if time permits.

At the mid-term stage, floating-point and fixed-point Python models of the full chain (transmitter, noisy channel, receiver, BER measurement) have been built and checked. The simulated BER follows the expected coherent-QPSK performance, the fixed-point model is close to the floating-point reference under the tested conditions, and golden vectors have been exported for later RTL verification. All results so far are from Python simulation. RTL and FPGA testing remain future work.

1. Introduction

Modern communication systems rely heavily on digital modulation to move information reliably and efficiently over a band-limited channel. Digital modulation schemes vary properties of a carrier, such as its amplitude, phase, or frequency, to represent information while making efficient use of the available communication bandwidth.

Quadrature Phase Shift Keying (QPSK) is one such scheme. Unlike Binary PSK, which uses two carrier phases to represent one bit per symbol, QPSK uses four equally spaced carrier phases, so that each transmitted symbol represents two bits at once [2]. This is conveniently described using an in-phase (I) and quadrature (Q) component, which together represent the two bits of each transmitted symbol. The transmitted signal can therefore be represented as a combination of cosine and sine carriers at the same frequency, with the I and Q components determining the corresponding symbol.

Field-Programmable Gate Arrays (FPGAs) provide reconfigurable digital hardware rather than a fixed instruction-execution pipeline: operations such as carrier generation, multiplication, and filtering can each be realized as dedicated, concurrently operating hardware blocks. This matters for a real-time communication system, where all stages of modulation and demodulation must run continuously at the signal's own rate. Prior work has already demonstrated that a complete QPSK modulator and demodulator can be realized on FPGA hardware [1, 5], which motivates implementing our system on FPGA rather than restricting it to a Python/MATLAB simulation alone.

Narrowing from this general motivation to our specific system, this project implements a complete QPSK transmit-receive chain in which a bit stream is first mapped to QPSK symbols, carriers are generated by an NCO, the symbols are modulated onto these carriers, and the resulting signal is passed through a simulated AWGN channel. At the receiver, the signal is coherently demodulated, passed through an FIR filter, and a symbol decision block recovers the original bit stream. Each of these stages is implemented as a distinct hardware block on the Spartan-7 FPGA (Boolean Board, XC7S50).

To validate this system, we follow a staged verification approach — software reference model, then fixed-point model, then FPGA/RTL implementation — comparing each stage against the previous one and evaluating the final system via BER, constellation diagrams, and FPGA resource utilization.

1.1. Problem Statement

The system described above runs entirely on one Boolean Board, with no external RF front-end, sensors or second board. The challenge is to build a signal-processing chain whose behaviour is verified quantitatively and which maps onto finite-precision FPGA hardware.

This raises three concerns:

1. **Functional correctness.** In the absence of noise, the receiver must recover every transmitted bit exactly. This depends on consistent bit ordering, sign conventions, and filter delay alignment between the transmitter and receiver.

2. **Noise performance.** The measured BER over a range of E_b/N_0 values should agree with the theoretical QPSK curve and with the analytical prediction for the implemented receiver.

3. **Hardware suitability.** Finite word lengths introduce quantization, overflow, and clipping. Bit widths, scaling, and filter structure must be chosen and checked before RTL implementation, because they can directly affect BER.

Our approach is staged: first a floating-point Python reference model, then a bit-accurate fixed-point model, and finally RTL on the FPGA, verified against test vectors exported from the Python models.

1.2. Objectives

The main objectives of this project are:

1. Implement a QPSK mapper and a transmitter based on a phase-accumulator numerically controlled oscillator (NCO) with a sine/cosine lookup table.

2. Model an AWGN channel with configurable E_b/N_0, using both a NumPy Gaussian reference and a hardware-oriented linear-feedback shift register (LFSR)-based noise generator.

3. Implement coherent I/Q downconversion, finite impulse response (FIR) low-pass filtering, integrate-and-dump, and sign-based symbol decisions.

4. Measure BER over an E_b/N_0 sweep and compare it with theory and with the analytical prediction for the implemented receiver.

5. Develop a fixed-point model and export hexadecimal golden vectors for RTL verification.

6. Implement the verified design on the Boolean Board and demonstrate it live.

1.3. Assumptions

The core system assumes a known carrier phase and known symbol timing at the receiver, with the local carrier locked to the transmitter. It is a classical design with no machine-learning component.

2. Literature Review

The theoretical foundation for this project is Proakis and Salehi [6], whose treatment of digital modulation and optimum receiver design for additive white Gaussian noise (AWGN) channels provides the signal-space and constellation basis for QPSK, along with the standard framework for coherent detection and error-rate analysis. This underlies the QPSK constellation, matched filtering, and BER evaluation used throughout our design, without being an FPGA implementation reference itself.

The most directly relevant prior FPGA implementation is Popescu *et al*. [5], who implement a complete QPSK modulator, channel, and demodulator rather than a modulator alone, using two Spartan-3E boards with ROM-generated sine/cosine carriers and coherent multiply-and-threshold detection at the receiver. This establishes that a complete FPGA-based QPSK transmit-receive chain is practical on modest hardware, and is the closest architectural match to our own single-board system.

On the modulator side, Al Safi and Bazuin [1] focus on carrier generation, producing the sine/cosine waveforms needed for BPSK/QPSK using a single LUT addressed by a reverse-accumulator technique to reduce memory — directly relevant to our own phase-accumulator-driven NCO. Pareek [4] shows that this I/Q-and-multiplier structure is not the only option, comparing it against a digitized-carrier approach that uses a 4:1 multiplexer to select one of four precomputed phase-shifted carriers directly from the input bit pair. We adopt the conventional I/Q route used by the first two papers, for its direct correspondence between modulation and demodulation stages.

On the demodulator side, Zhang *et al*. [10] present an all-digital receiver built from digital down-conversion (NCO, mixer, low-pass filter), a four-branch Costas loop for carrier synchronization, and a digital-phase-locked-loop bit synchronizer. This is useful primarily as a contrast: it shows what a synchronization-capable receiver requires, which lets us state plainly that our baseline demodulator instead assumes a fixed, known carrier phase and symbol timing, deferring carrier/timing recovery to the optional-extension scope described in section 1.

Deepika *et al*. [7] demonstrate that coherent I/Q multiplication is not the only demodulation strategy either, proposing a comparator-based QPSK demodulator built with model-based design and implemented on a Zynq-7000 (ZED) board, with reported hardware resource utilization.

This confirms that alternative, lower-complexity demodulator architectures exist, though our project uses explicit I/Q demodulation with locally generated references, FIR filtering, and symbol decision rather than a comparator-based scheme.

Taken together, the reviewed literature establishes the theoretical basis for QPSK and AWGN detection, demonstrates a complete two-board FPGA QPSK system, and covers multiple approaches to both carrier generation/modulation and demodulation, ranging from ROM/accumulator carrier generation to comparator-based and synchronization-capable receivers. Our goal is the integration of established, conventional techniques into one single-FPGA system with a digitally simulated, live-adjustable AWGN channel, BER measurement, and UART-streamed constellation visualization.

2.1. Further Literature Review (Mid-Term)

2.1.1 FPGA-Based QPSK Modem Architectures

Field-programmable gate arrays (FPGAs) allow digital communication systems to be organized into separate, reusable processing blocks. Mandadkar et al. [3] describe a QPSK modulator and demodulator intended for software-defined radio. Their architecture includes an I-Q mapper, carrier-generation blocks, multipliers, and decision circuits. The paper discusses modelling the system in MATLAB Simulink and implementing its modules using VHDL on an FPGA platform.

A QPSK transmitter can be represented as two data branches that modulate orthogonal carriers. The in-phase and quadrature components are then combined to form the transmitted waveform. This block-based representation supports staged development, since the mapper, carrier generator, modulator, and receiver can be checked individually before integration. This is relevant to the modular approach used in the present project.

2.1.2 DDS and NCO-Based Carrier Generation

Song and Yao [9] present an FPGA-based QPSK modem that uses direct digital synthesis (DDS) to generate orthogonal carrier signals. In a DDS/NCO structure, a phase accumulator is updated by a frequency control word, and a lookup table (LUT) maps the phase to digital sine or cosine samples. For an accumulator of width M, the output frequency can be expressed as

$$f_{\text{out}} = \frac{K f_{\text{clk}}}{2^M}, \tag{1}$$

where K is the frequency control word and f_{clk} is the clock frequency.

Song and Yao also describe low-pass FIR filtering in the receiver after carrier multiplication. Their paper reports simulation and FPGA implementation using the Quartus II design environment and a Cyclone II device, while noting that synchronization is not included in their modem. The DDS structure is directly relevant to this project, whose carrier is generated using a phase accumulator and a LUT. The accumulator width controls phase resolution, while the LUT address width determines how many stored phase samples can be selected.

2.1.3 FIR Filtering and Coherent Receiver Processing

In coherent QPSK demodulation, the received waveform is multiplied by local in-phase and quadrature carriers to move the data-bearing components to baseband. The multiplication also produces higher-frequency components, which can be attenuated by low-pass filtering. Song and Yao [9] include FIR filters in both receiver branches, while Mandadkar et al. [3] describe quadrature demodulation followed by decision circuits.

The receiver in the present project follows the same general structure, with I/Q downconversion, optional FIR filtering, integrate-and-dump processing, and sign-based decisions. The FIR filter introduces a group delay that must be considered when selecting the symbol-decision timing. Filtering and timing alignment can therefore affect BER even when the carrier frequency and phase are assumed to be known.

2.1.4 LUT Optimization and FPGA Resource Use

Sakran et al. [8] investigate a DDS-based QPSK modulator that reduces the number of stored carrier samples by using a smaller LUT and phase-related logic to reconstruct the required waveforms. Their paper reports lower LUT and flip-flop use for their proposed design than for the full-table comparison used in that work. Those resource figures depend on their design and target platform and should not be treated as expected results for the Spartan-7 board used here.

The paper highlights a general implementation trade-off: LUT size, phase representation, and reconstruction logic influence the hardware resources required to generate the carrier. The present project uses an eight-bit LUT address derived from a 16-bit phase accumulator. A reduced-LUT design could be investigated later, but it would need to be checked for waveform accuracy and compatibility with the receiver.

2.1.5 Relevance to the Present Project

The reviewed papers provide background for three aspects of this project: modular QPSK modem design, DDS/NCO carrier generation with lookup tables, and receiver filtering and hardware resource considerations. The current work

uses these ideas as a basis for a staged verification process. A floating-point Python model is used as a reference, a fixed-point model is used to study finite-width arithmetic, and golden vectors have been exported for future RTL testing. The project has not yet reproduced the FPGA resource figures of the reviewed papers; the current quantitative results focus on Python-model correctness and BER.

3. System Architecture and Methodology

The design is developed in three stages: a floating-point Python reference model, a bit-accurate fixed-point Python model, and (planned) RTL for the Spartan-7 XC7S50 on the Boolean Board. This section describes the first two stages. The RTL stage has not started at the time of writing.

3.1. Signal Flow

Figure 4 (the block diagram from the preliminary report) shows the complete chain. The transmitter maps bits to I/Q symbols and modulates an NCO carrier. The channel adds white Gaussian noise to the transmitted samples. The receiver multiplies by local carrier references, low-pass filters the products, integrates over each symbol and decides the bits from the signs. The receiver assumes perfect knowledge of the carrier phase and symbol timing (Section 1.3).

3.2. Model Parameters

Table 1 lists the settings used in the version of the code reported here (`params.py` and `rx_frontend.py`). If any of these settings change later, every number in this report has to be rerun.

Parameter	Value
Samples per symbol N_s	16
Carrier cycles per symbol	2 ($f_c/f_s = 0.125$)
NCO accumulator / LUT address	16 bit / 8 bit (256 entries)
NCO phase increment	8192
Datapath (LUT, carrier)	12 bit signed, full scale 2047
TX mixer output	13 bit signed
RX input word	12 bit signed, full scale ± 8.0
RX mixer output	12 bit signed (saturating)
FIR	25 taps (order 24), Hamming-windowed sinc
FIR cutoff (-6 dB)	0.125 cycles/sample
FIR coefficients	12 bit, Q1.11, DC gain exactly 1
FIR group delay	12 samples
Integrate-and-dump accumulator	16 bit signed

Table 1. Parameters of the Python golden model.

3.3. Bit Source and Mapper

Two bit sources are used. BER sweeps use NumPy random bits, so the statistics are not tied to a short repeating pattern. The hardware-oriented source is a 7-bit Fibonacci LFSR (PRBS-7, seed `0x7F`, feedback = bit 0 $\oplus$ bit 6, shift right), whose period is $2^7 - 1 = 127$ bits; the Python model reproduces the intended Verilog update bit for bit, so the same sequence can be generated in RTL.

Bit pairs (b_0, b_1) are mapped to symbols by

$$I = 1 - 2b_0, \qquad Q = 1 - 2b_1, \tag{2}$$

so a 0 bit becomes $+1$ and a 1 bit becomes -1 on each branch. Writing the pair as (I, Q), this gives $00 \to (+1, +1)$, $01 \to (+1, -1)$, $11 \to (-1, -1)$ and $10 \to (-1, +1)$. The four points sit at $45°$, $315°$, $225°$ and $135°$. Going around the circle, neighbouring points differ in only one bit (Gray coding). The most likely mistake is landing in a neighbouring region, and with this mapping that costs one bit error instead of two.

3.4. Numerically Controlled Oscillator

The carrier is produced by a phase accumulator and a sine/cosine lookup table (LUT). At every sample the 16-bit accumulator adds the increment

$$\Delta\phi = \mathrm{round}\left(\frac{f_c}{f_s}\, 2^{16}\right) = 8192, \tag{3}$$

and its upper 8 bits address a 256-entry LUT holding $\mathrm{round}(2047\cos\theta)$ and $\mathrm{round}(2047\sin\theta)$ as 12-bit two's-complement integers. Because $\Delta\phi$ is a multiple of $2^{16-8} = 256$, every visited phase lands exactly on a LUT entry, so phase truncation introduces no error for this carrier. The NCO then visits only 8 of the 256 LUT points (multiples of $45°$), with values $\{\pm 2047, \pm 1447, 0\}$, and returns to phase zero after every symbol (two full cycles in 16 samples). The fixed-point transmitter exploits this by tiling a single symbol of NCO output.

3.5. QPSK Modulator

The transmitted samples are

$$s[n] = I[n]\cos(\omega_c n) - Q[n]\sin(\omega_c n), \tag{4}$$

with each I, Q value held for $N_s = 16$ samples. The minus sign on the Q branch is a fixed convention shared by transmitter and receiver; changing it in only one place inverts every Q decision. In the integer model, with $I, Q \in \{\pm 1\}$ the product is exact, and the worst case $2047\sqrt{2} \approx 2895$ exceeds the 12-bit range, so the mixer output is 13 bits wide. The model raises an exception instead of wrapping if a value would not fit.

3.6. AWGN Channel

The received samples are $r[n] = s[n] + w[n]$ with white noise $w[n]$ of variance σ^2. For the unit-amplitude carrier the mean signal power is $P = 1$, so the symbol energy is $E_s = PN_s$ and, since QPSK carries two bits per symbol, $E_b = E_s/2$. With $\sigma^2 = N_0/2$ per real sample,

$$\sigma^2 = \frac{P\,N_s}{4\,(E_b/N_0)}, \tag{5}$$

where E_b/N_0 is a linear ratio and P is measured from the signal, so the same function serves the float and the rescaled integer signal.

Two noise sources are implemented. The reference source is NumPy's standard normal generator. The hardware-style source is built from a 16-bit LFSR (feedback taps at bits 0, 2, 3 and 5, period $2^{16} - 1 = 65535$): eight successive bits form one byte, and six byte streams from different seeds are summed and normalised to zero mean and unit variance, which approximates a Gaussian distribution by the central limit theorem. Over 10^5 samples this gave a measured mean of -0.002 and variance of 1.001. It is an approximation rather than true Gaussian noise: the stream repeats every 65535 samples (about 4096 symbols), the six streams are different phases of the same sequence, and the sum of six bytes has bounded support, so the noise is truncated at about $\pm 4.2\sigma$. The LFSR source is therefore used only for short runs; all BER sweeps use the NumPy source.

3.7. Receiver Front End

The receiver multiplies the received signal by the local carrier,

$$x_I[n] = r[n]\cos(\omega_c n), \qquad x_Q[n] = -r[n]\sin(\omega_c n). \tag{6}$$

Each product contains the wanted data ($I/2$ or $Q/2$) at baseband and an unwanted component at $2f_c$. The minus sign in x_Q cancels the minus sign used in the transmitter.

A 25-tap low-pass FIR filter (Hamming window, cutoff 0.125 cycles/sample) removes the $2f_c$ component on each branch. In the floating-point design, the gain stays within 0.1 dB of unity in the data band ($f \le 1/16$) and is more than 40 dB down for $f \ge 3/16$, where the $2f_c$ component lies.

The filter is causal, so its output is delayed by $D = 12$ samples. We handle this in one of two ways: either read symbol k starting at sample $16k + D$, or shift the output back so that no offset is needed. Both give identical decision values (Section 3.10).

The fixed-point version uses only integers, as the hardware will. The received sample is rounded to a 12-bit word with full scale ± 8.0, so the transmit amplitude 1.0 becomes 256. This range covers $\pm 4\sigma$ even at $E_b/N_0 = 0$ dB, where $\sigma = 2$. The mixer and the FIR both compute an exact product (or sum of products), add 2^{10}, shift right by 11 bits and saturate to 12 bits. The 12-bit filter coefficients use Q1.11 format and are adjusted to add up to exactly 2048, so the gain at 0 Hz is exactly 1.

3.8. Integrate-and-Dump and Decision

Each symbol is recovered by summing 16 consecutive filter outputs,

$$\hat{I}_k = \sum_{m=0}^{15} y_I[16k + m], \tag{7}$$

$$\hat{Q}_k = \sum_{m=0}^{15} y_Q[16k + m], \tag{8}$$

which is the matched filter for a rectangular pulse. Because the $2f_c$ product completes exactly four cycles in 16 samples, the sum removes it completely even without the FIR. For a noiseless symbol, $\hat{I}_k = \pm 8$ (baseband level $\pm 0.5 \times 16$). The bit decision is

$$\hat{b}_0 = \begin{cases} 0, & \hat{I}_k \ge 0 \\ 1, & \hat{I}_k < 0 \end{cases} \tag{9}$$

and the same rule is used for $\hat{b}_1$ with $\hat{Q}_k$. A value of exactly zero is decided as bit 0. In the integer model the accumulator needs $12 + \log_2 16 = 16$ signed bits. When counting errors the first and last symbols are skipped, because they see filter start-up and cut-off.

3.9. BER Analysis

For ideal coherent QPSK in AWGN the reference curve is

$$P_b = Q\left(\sqrt{2E_b/N_0}\right). \tag{10}$$

For the actual receiver (mixer, FIR, 16-sample integration) the model also computes an exact prediction without random simulation. The decision value of a symbol depends only on that symbol and its neighbours, so the noiseless chain is evaluated for all $4^3 = 64$ three-symbol neighbourhoods to obtain the signal margin d. The decision-noise variance follows in closed form from the combined kernel $g = \mathrm{box}_{16} * h$, and the BER is the average of $Q(d/\sigma_d)$ over the neighbourhoods. Simulated results are compared with the matching prediction using a z-score

$$z = \frac{N_{\mathrm{err}} - Np}{\sqrt{Np(1-p)}}, \tag{11}$$

and the number of simulated bits at each E_b/N_0 is chosen so that about 100 errors are expected (capped at 1.5×10^6 bits).

3.10. Verification Procedure

We check the model in stages, so that each check looks for one kind of mistake. The noiseless test comes first. If the chain cannot recover the bits with no noise, something is wrong with the mapping, the signs or the bit order, and no BER number should be trusted. Next we check the filter delay: the causal FIR output, read with a 12-sample offset, must give the same decision values as the aligned output. Then we run BER sweeps and compare them with the QPSK formula and with the exact prediction for our FIR receiver. After that we run the integer (fixed-point) chain on the same bits and noise as the floating-point chain and compare the two. Last, we run a short test with the LFSR noise source and export the golden vectors. All of these checks are `assert` statements in the scripts, so a failed check stops the run. Table 2 summarises them and their outcome.

Check	Purpose and outcome
Noiseless end-to-end test	Zero bit errors for the tested floating-point and fixed-point receiver variants.
FIR delay alignment	The causal FIR decisions agree with the aligned FIR decisions for 499 of 500 compared symbols when the 12-sample delay is accounted for.
BER sweep	The no-FIR BER agrees with theoretical coherent-QPSK BER; the FIR BER agrees with the prediction for the implemented FIR receiver within the observed statistical variation.
Fixed-point comparison	At 4 dB, the fixed-point and floating-point BER values are 0.01436 and 0.01435, respectively, using the same bits and noise.
LFSR noise test	At 2 dB, the LFSR-based run gives BER 0.0391 compared with a model prediction of 0.0415.
Golden-vector export	The exported files are read back and checked against the arrays written by the Python script.

Table 2. Summary of the current Python-model verification checks. These checks do not represent RTL or FPGA verification.

3.11. Golden Vectors for RTL

The fixed-point script exports six hexadecimal files for later verification of the integrate-and-dump and decision stages. The files contain transmitted bits, filtered I and Q samples, the I and Q accumulator values, and received bits. The export covers 127 symbols, uses a 12-sample timing offset and 16-bit signed accumulators, and passes through the LFSR-based noise channel at 6 dB. The comparison contains 250 bits and produces zero errors for this test. The files are read back and checked against the arrays that were written. They are reference inputs and outputs for a future RTL testbench; their generation does not show that any RTL has passed.

4. Results Replicated from the Literature

4.1. Theoretical BER of coherent QPSK

A standard reference result for coherent QPSK in an additive white Gaussian noise (AWGN) channel is the theoretical bit-error rate (BER) [6]:

$$P_b = Q\left(\sqrt{\frac{2E_b}{N_0}}\right), \tag{12}$$

where E_b/N_0 is the bit-energy-to-noise-density ratio and $Q(\cdot)$ is the Gaussian tail probability. This result is used as the analytical reference for the present simulations. With rectangular symbol pulses and ideal carrier and timing alignment, the receiver without the FIR uses mixing followed by a 16-sample integrate-and-dump operation. Its simulated BER is expected to follow Equation 12.

Table 3 shows that the no-FIR results are close to the theoretical values at all five tested points. The normalized differences are within $|z| \leq 1.5$ for this run, consistent with the statistical variation expected from a finite number of transmitted bits. The comparison also provides a check of the implemented noise scaling. Figure 1 shows the same comparison on a log scale, with the simulated points sitting on the curves. The FPGA papers we reviewed focus mainly on architecture and device-specific implementation results, so this report does not claim to reproduce their particular FPGA resource counts or hardware measurements.

4.2. DDS frequency relation

Song and Yao [9] give the output frequency of a DDS/NCO as $f_{\text{out}} = K f_{\text{clk}}/2^M$ (Eq. 1). We checked our NCO against it. With $M = 16$ and $K = 8192$ the formula gives $f_{\text{out}} = f_{\text{clk}}/8$, that is $f_c/f_s = 0.125$, or two carrier cycles in each 16-sample symbol, as intended. In our simulated NCO the output repeats every 8 samples. The cosine goes $2047, 1447, 0, -1447, -2047, -1447, 0, 1447$, and the sine is the same list delayed by two samples. These are the stored 12-bit values of cosine and sine at multiples of $45°$. The only difference from an ideal sinusoid is rounding to whole numbers, at most half a code (for example, $2047 \cos 45° = 1447.4$ is stored as 1447), which is under $0.03\,\%$ of full scale. We built the NCO with the same structure as the paper, so this agreement is expected. It confirms that our implementation is correct; it is not a new result.

4.3. QPSK signal and constellation

The NPTEL notes [2] write the QPSK signal as $s_i(t) = \sqrt{2E/T}\cos\big(2\pi f_c t + (2i-1)\pi/4\big)$ for $i = 1, \ldots, 4$, so

the four symbols sit at $45°$, $135°$, $225°$ and $315°$, and the signal behaves like two BPSK signals on orthogonal carriers. Our transmitted signal can be rewritten as $I\cos\omega_c n - Q\sin\omega_c n = \sqrt{2}\cos(\omega_c n + \phi)$ with $\phi = \text{atan2}(Q, I)$. For our mapping this gives exactly the same four phases: $(+1,+1)$ at $45°$, $(-1,+1)$ at $135°$, $(-1,-1)$ at $225°$ and $(+1,-1)$ at $315°$. The receiver constellations in Fig. 2 show four clusters around $(\pm1, \pm1)$, as expected. The notes also describe a correlation receiver that multiplies by the two carriers and integrates over a symbol. Our mixer followed by integrate-and-dump is the sampled version of that receiver.

5. Preliminary Results

5.1. Noiseless Correctness

With noise disabled, the floating-point chain gives zero bit errors in the tested receiver variants: no FIR, 0/3996 bits; aligned FIR, 0/3996 bits; and causal FIR, 0/3994 bits. For the no-FIR case, the decision values are exactly $\pm8 = \pm N_s/2$, where $N_s = 16$ samples per symbol. The fixed-point chain also gives zero errors for all three variants. In the noiseless fixed-point test, the maximum accumulator magnitudes are 2048 without the FIR and 2132 for the tested FIR path.

When the FIR delay is handled using an offset of $D = 12$ samples, all 499 complete symbols contained within the causal FIR evaluation window match the zero-delay aligned FIR decisions with zero errors. The 500th symbol extends past the end of the finite 8000-sample array due to the 12-sample shift and is truncated, rather than representing a decision mismatch. Using no offset does not give the same result and degrades decision values across symbol boundaries, confirming that the FIR group delay must be accounted for when locating symbol-decision intervals.

5.2. BER versus E_b/N_0

Table 3 shows the integer-chain BER with and without the FIR, alongside theoretical coherent-QPSK BER and the prediction for the implemented FIR receiver. Each point uses approximately 100,000 bits, except the 8 dB point, which uses 599,976 bits. BER decreases as E_b/N_0 increases. Each measured value lies within $|z| \leq 1.9$ of its corresponding prediction for this run.

The FIR does not improve BER in the present receiver arrangement. In the floating-point model, its estimated penalty rises from about 0.215 dB at 0 dB to about 0.337 dB at 8 dB. One possible explanation is that the FIR spreads symbol transitions, while the 16-sample integrate-and-dump operation already suppresses the doubled-carrier component under the assumed alignment. The filter is retained because it is part of the planned receiver architecture, not because it improves the current simulated BER. At 8 dB, the simulated BER values are below their predictions; the number of errors is relatively small, so a longer run would provide a more precise comparison.

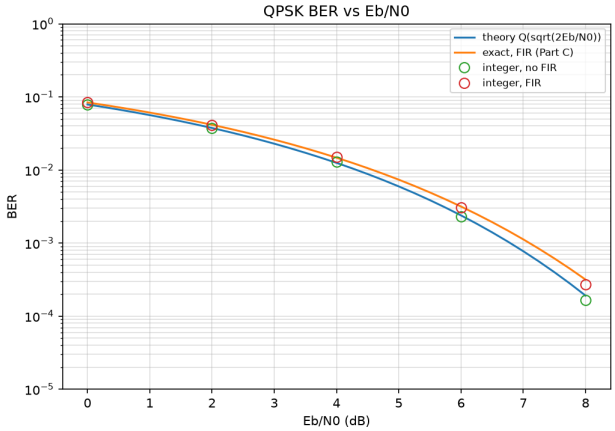


Figure 1. BER versus E_b/N_0 for the fixed-point (integer) chain, with and without the FIR, compared with the theoretical QPSK curve and the exact prediction for the FIR receiver.

5.3. Constellation Plots

Figure 2 shows the receiver decision values at 0, 6, and 12 dB. The four clusters become more compact as E_b/N_0 increases. At 0 dB, the clusters spread across the decision boundaries, which is consistent with the higher BER. The values in the plot are scaled for display; the raw accumulator outputs are not limited to ±1.

5.4. Fixed-Point versus Floating-Point Model

At 4 dB, using the same transmitted bits and noise, the fixed-point chain gives a BER of 0.01436, while the floating-point chain gives a BER of 0.01435. Figure 3 compares the BER performance of the fixed-point and floating-point implementations, with and without FIR filtering, across different E_b/N_0 values. The results closely overlap, indicating that fixed-point quantization introduces only a small difference in BER performance. The RMS difference between their decision values is 0.0009 on a normalized noiseless level of 1. The input clipping fraction is zero, and the maximum accumulator magnitude is 6066, compared with the 16-bit signed limit of 32767. Therefore, no accumulator overflow occurred in this test. However, since this comparison was performed at a limited number of noise levels, it should not be taken as proof that the two implementations agree equally well at every E_b/N_0.

E_b/N_0 (dB)	No FIR	Theory	FIR	FIR prediction
0	7.831×10^{-2}	7.865×10^{-2}	8.400×10^{-2}	8.385×10^{-2}
2	3.745×10^{-2}	3.751×10^{-2}	4.120×10^{-2}	4.145×10^{-2}
4	1.301×10^{-2}	1.250×10^{-2}	1.500×10^{-2}	1.472×10^{-2}
6	2.320×10^{-3}	2.388×10^{-3}	3.070×10^{-3}	3.159×10^{-3}
8	1.650×10^{-4}	1.909×10^{-4}	2.717×10^{-4}	3.162×10^{-4}

Table 3. Simulated BER of the integer chain compared with theoretical QPSK BER and the prediction for the implemented FIR receiver.

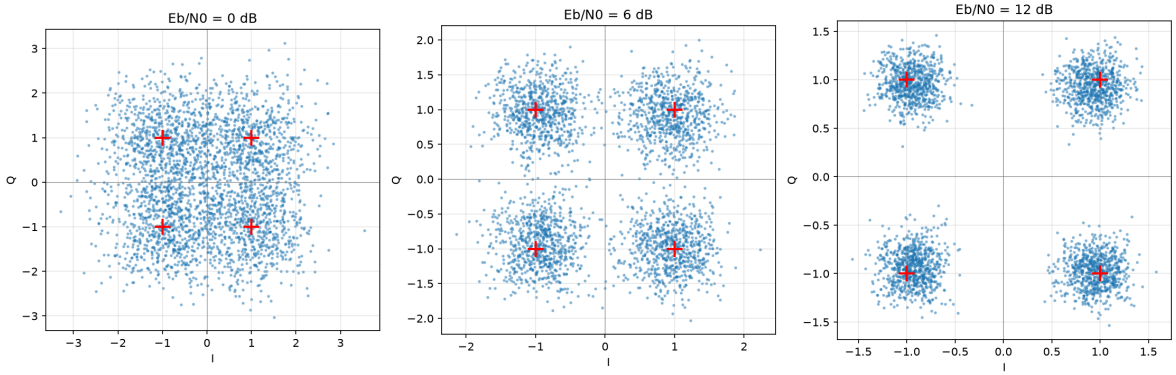


Figure 2. Receiver constellations of the fixed-point chain at 0, 6, and 12 dB. The red crosses mark the ideal points $(\pm 1, \pm 1)$.

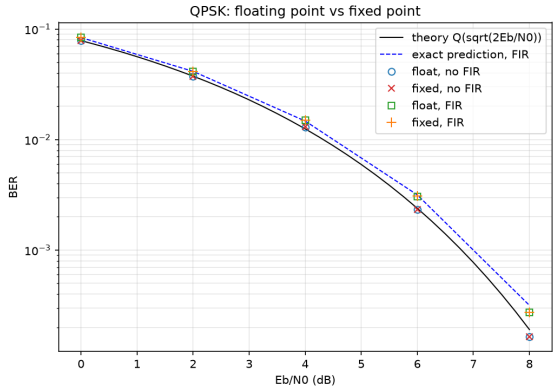


Figure 3. BER performance of the fixed-point and floating-point QPSK implementations, with and without FIR filtering, compared with the theoretical BER curve.

5.5. LFSR-Based Noise Test

Using the hardware-oriented LFSR noise source, a run of 4000 symbols at 2 dB gives BER 0.0391 compared with a model prediction of 0.0415. The measured value is approximately one standard deviation below the prediction and is consistent with the expected variation for this run. The LFSR sequence is deterministic and periodic, so it is treated as an approximation for hardware-oriented testing. The BER sweeps in Table 3 use NumPy-generated Gaussian noise instead.

6. Next Steps

Having established the problem statement and reviewed relevant prior work, the next phase of this project is to design, implement, and verify the core QPSK system described in section 1. We plan to follow a staged verification approach: a software reference model will be developed first in Python, followed by a fixed-point version of the same algorithm, and finally the FPGA/RTL implementation on the Boolean Board. At each stage, the implementation's output will be compared against the previous stage to isolate errors early, rather than debugging the algorithm and the hardware simultaneously.

Concretely, for the mid-term report we aim to have a working modulator and a demodulator that assumes a fixed, known carrier phase, verified in simulation, along with the simulated AWGN channel with its live-adjustable noise level. For the final report, we aim to complete the bit-error-rate measurement pipeline and the live constellation visualization streamed over UART, and to demonstrate the full system on the Boolean Board.

If time permits after this core system is verified and working, we will consider the optional extensions introduced in section 1: carrier phase recovery via a Costas loop, symbol timing recovery via the Gardner algorithm, and a coded-versus-uncoded BER comparison. These remain secondary to completing and validating the core system.

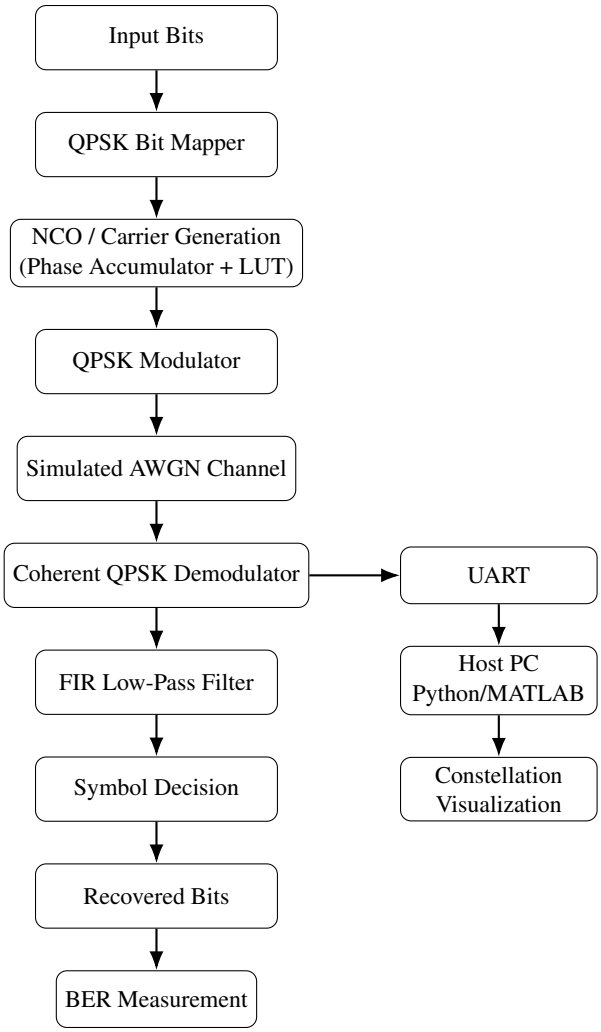


Figure 4. QPSK transmitter–channel–receiver block diagram

Update at the mid-term stage. The Python reference and fixed-point models described in the sections above are now complete. For the remaining work: The next stage is to implement and simulate the NCO, mapper, transmitter mixer, receiver mixer, FIR filter, integrate-and-dump logic, and decision logic in Verilog. The RTL outputs will be compared with the exported golden vectors. After block-level verification, the design will be integrated on the RealDigital Boolean Board with the Spartan-7 XC7S50, and the LUT, flip-flop, DSP, and memory use will be recorded. If supported by the remaining time, a UART-based constellation output and hardware BER measurements will be added and compared with the Python reference model. Optional extensions include carrier recovery using a Costas loop, timing recovery using a Gardner detector, and a comparison of coded and uncoded BER.

7. AI Usage

We used AI in the following ways. We have gone through all of this content and can explain it.

- **Golden vectors:** AI helped us write the code that exports the test vectors (the hex files) for the future Verilog testbench.
- **Receiver front-end code:** AI helped with some functions in the front-end code, such as the fixed-point mixer, the FIR filter and the BER analysis. We reviewed these and tested them against the floating-point version.
- **NCO:** AI gave some help with the NCO code in the TX NCO file, mainly the phase accumulator and the sine/cosine lookup table.
- **Main code:** Also AI was used to identify suitable outputs, performance metrics, and visualizations to include.
- **LFSR-based noise:** AI helped us build the hardware-friendly Gaussian noise generator from LFSRs, which we can later implement in Verilog.
- **Explaining results and concepts:** AI helped us understand and explain the results.

8. Conclusion

This report presented the mid-term state of our QPSK modem for the Boolean Board. So far, the transmitter, noisy channel, receiver and BER measurement exist as floating-point and fixed-point Python models.

With no noise, the chain recovers every bit. With noise, the simulated BER matches the coherent-QPSK formula for the receiver without the FIR, and matches our exact prediction for the receiver with the FIR, within normal statistical variation. The FIR costs about 0.2 to 0.34 dB of E_b/N_0, so it does not improve BER in the current setup.

The fixed-point model gives almost the same BER as the floating-point model (0.01436 against 0.01435 at 4 dB), with no clipping or overflow. The LFSR noise test also gave a BER close to the prediction, and golden vectors for 127 symbols were exported for later testing.

All of these are Python results. No Verilog has been simulated and nothing has been run on the FPGA yet. The next step is to write and simulate the RTL blocks, compare them with the golden vectors, and then bring the design up on the Boolean Board.

References

[1] Amean Al Safi and Bradley Bazuin. Fpga based implementation of bpsk and qpsk modulators using address reverse accumulators. In *2016 IEEE 7th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, pages 1–6, 2016. 1, 2

[2] ECE, IIT Kharagpur. Quaternary phase shift keying (qpsk) modulation. NPTEL, Module 5, Lesson 25. Version 2. 1, 6

[3] Mukesh Mandadkar, Abhishek Lokhande, and R. R. Bhambare. Qpsk modulator and demodulator using fpga for sdr. *International Journal of Engineering Research and Applications (IJERA)*, 4(4):394–397, April 2014. 3

[4] Vishakha Pareek. A novel implementation of qpsk modulator on fpga. In *2016 IEEE Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, pages 1–4, 2016. 2

[5] S.O. Popescu, A.S. Gontean, and D. Ianchis. Implementation of a qpsk system on fpga. In *2011 IEEE 9th International Symposium on Intelligent Systems and Informatics*, pages 365–370, 2011. 1, 2

[6] John G. Proakis and Masoud Salehi. *Communication Systems Engineering*. Prentice Hall / Pearson Education, 2nd edition, 2002. 2, 6

[7] Deepika R., Sourabh K.S., and Manikandan J. Design and implementation of a novel qpsk demodulator for fpga based system design. In *2019 IEEE International WIE Conference on Electrical and Computer Engineering (WIECON-ECE)*, pages 1–4, 2019. 2

[8] Rawaa Kadhim Sakran, Riham Ali Zubaid, and Ahmed Abdulredha Ali. Design and implementation of QPSK modulator using FPGA based on DDS algorithm. *University of Thi-Qar Journal for Engineering Sciences*, 15(1), 2025. 3

[9] Wenmiao Song and Qiongqiong Yao. Design and implement of qpsk modem based on fpga. volume 9, pages 599 – 601, 08 2010. 3, 6

[10] Huiying Zhang, Ting Zhao, and Ruijing Xie. Research of all digital demodulation technology for qpsk. 01 2015. 2